

Servidor Web de Arquivos Estáticos (J-WebServer)

1. Visão Geral

Este projeto desafia o grupo a mergulhar na infraestrutura da Internet, construindo do zero um **Servidor Web funcional em Java**. O objetivo não é apenas criar um site, mas sim o *software* que entrega o site. A aplicação deve ser capaz de escutar requisições de navegadores (como Chrome ou Firefox), interpretar o protocolo HTTP e servir arquivos (HTML, CSS, Imagens) armazenados localmente, mantendo um controle rigoroso de acessos e segurança.

2. Objetivos de Aprendizado e Aplicação (POO)

Este projeto é focado no núcleo de "baixo nível" do Java e na robustez do sistema, aplicando conceitos avançados do plano de ensino:

- **Manipulação de Threads e Concorrência:** Para que o servidor não trave ao atender um usuário, a equipe deve utilizar *Threads* para gerenciar múltiplas conexões simultâneas via `ServerSocket`.
- **Encapsulamento e Segurança:** Utilizar modificadores de acesso (`private`, `protected`) para garantir que o servidor só acesse arquivos dentro de pastas permitidas, evitando vulnerabilidades de segurança (como o acesso a arquivos sensíveis do sistema operacional).
- **Tratamento de Exceções:** Implementar uma estratégia robusta de `try-catch`. O servidor deve ser resiliente: se ocorrer um erro ao gravar um log no banco de dados (`SQLException`), a entrega do arquivo ao usuário não deve ser interrompida.

3. Requisitos de Persistência (JDBC)

O servidor deve atuar como uma ferramenta de auditoria de TI, utilizando os conhecimentos:

- **Database Logger (DAO):** Cada vez que um arquivo é solicitado, o sistema deve registrar no banco de dados o endereço IP do cliente, a data/hora, o recurso acessado e o código de status (ex: 200 OK ou 404 Not Found).
- **Persistência de Erros:** Armazenar logs de tentativas de acesso não autorizadas para análise posterior de segurança.

4. Bônus: Interface Gráfica e Monitoramento (JavaFX)

Diferente de um servidor que roda apenas no terminal, este projeto exige um console de gerenciamento visual:

- **Monitor UI:** Uma interface JavaFX que exibe o status atual do servidor (Ligado/Desligado), o número de usuários conectados no momento e um fluxo em tempo real (Log Viewer) das requisições processadas.
- **Organização de Layout:** Uso de `VBox` para o console de logs e botões de controle para iniciar ou parar o serviço de rede.

5. Divisão de Responsabilidades (Equipe de 4 Alunos)

Para garantir a entrega em 8 semanas, a equipe deve se organizar nos seguintes papéis:

1. **Network Engine:** Responsável pelo gerenciamento de `ServerSocket` e fluxo de entrada/saída de dados brutos.
2. **HTTP Parser:** Responsável pela lógica de interpretação de métodos (GET, POST) e tratamento de cabeçalhos das requisições.
3. **Database Logger:** Implementação do padrão DAO para registrar logs de IP, data e recursos acessados.
4. **Security & Auth:** Gestão de modificadores de acesso e validação rigorosa de permissões de arquivos no disco.

6. Entregas Esperadas

Ao final a equipe deverá apresentar:

1. **O Servidor em Execução:** Abrir um navegador, digitar `localhost:8080` e visualizar uma página HTML sendo servida pelo programa Java.
2. **Relatório de Banco de Dados:** Demonstrar a tabela de logs preenchida com o histórico de navegação capturado via JDBC.
3. **Código-Fonte:** Organizado com foco na separação entre a lógica de rede (backend) e a interface de monitoramento (frontend).